

1. ДРАКОН-схемы

ДРАКОН-схема — это графический метод, разработанный для визуализации алгоритмов и процессов. Она помогает понять и представить логику программирования и бизнес-процессов.

Ключевые моменты:

- Иконы языка:

	Икона	Название иконы	Пояснение
1		Заголовок	В иконе «Заголовок» пишут полное название алгоритма, например: «Аварийное выключение ракетного двигателя»
2		Конец	В этой иконе пишут слово «Конец»
3		Действие	Указывают действие или команду, которую должен выполнить компьютер или человек
4		Вопрос	Да-нетный вопрос, т. е. вопрос, на который можно ответить либо Да, либо Нет. Все другие ответы запрещены
5		Выбор	Фраза (или вопрос), приглашающая выбрать один из вариантов
6		Вариант	Здесь пишут один из вариантов. (Число рассматриваемых вариантов равно числу икон «Вариант»)
7		Имя ветки	Эта икона обозначает начало ветки. В ней находится название ветки. (Ветка — это структурная часть алгоритма)
8		Адрес	Икона «Адрес» обозначает конец любой ветки, кроме последней. Она показывает, в какую следующую ветку надо идти
9		Вставка	Икона «Вставка» говорит, что в этом месте из алгоритма вынут «кусочек», который перенесён в другое место. В иконе пишут название этого «кусочка»
10		Полка	Сверху пишут указание, например: «Установить признак», снизу пояснение, например: «Можно ехать налево»
11		Формальные параметры	Икона присоединяется справа к иконе Заголовок
		Пояснение	Любые сведения, поясняющие икону, находящуюся слева от данной
12		Начало цикла ДЛЯ	Для цикла for пишут переменную цикла, ее начальное и конечное значения, шаг. Иконы 12 и 13 используются также в цикле foreach
13		Конец цикла ДЛЯ	Внутри иконы пишут «Конец цикла». Запрещено оставлять икону пустой
14		Вывод	Вывод данных, передача информации. Выдача команд на исполнительные органы в системах реального времени
15		Простой вывод	Икона «Простой вывод» позволяет упростить дракон-схему при необходимости
16		Ввод	Ввод данных, прием информации
17		Простой ввод	Икона «Простой ввод» позволяет упростить дракон-схему при необходимости
18		Пауза	Пауза задерживает выполнение следующего действия. Время задержки пишут внутри иконы.
19		Период	Период повторения действия, которое выполняется в цикле ЖДАТЬ
20		Таймер	Если в иконе написано A = 0, это значит, что данная икона создает, обнуляет и запускает таймер A

	Икона	Название иконы	Пояснение
21		Синхронизатор	Останавливает процесс, ожидая появления заданного момента времени
		Время	Если в иконе Время написано «10 секунд», это значит, что действие в иконе справа следует выполнить за 10 секунд или меньше
22		Время группы	Длительность выполнения группы действий. Не одного действия, а именно группы. Группа состоит из двух и более действий
23		Время группы справа	Икона «Время группы» присоединяется справа
24		Начало контрольного срока	В иконе Начало пишут контрольное время критической процедуры. Например, «30 сек».
25		Конец контрольного срока	Указывают окончание контрольного времени. Например, «Прошло 30 сек».
26		Начало параллельных действий	Означает НАЧАЛО нескольких действий, которые могут начинаться и выполняться как синхронно, так и в любом порядке
27		Конец параллельных действий	Означает момент времени, когда закончилось самое последнее действие из числа параллельных действий
28		Параллельный процесс	Осуществляет пуск, останов, приостановку и рестарт параллельного процесса
29		Комментарий	Комментарий — это не действие. Это различные пояснения и подсказки, которые помогают быстрее понять алгоритм
30		Выноска	Выноска содержит комментарий и может присоединяться к любой иконе с любой стороны
31		Соединитель	Икона «Соединитель» используется при переходе с листа на лист (когда алгоритм размещается на нескольких листах). В кружке пишут номер соединителя
32		Событие	Икона «Событие» останавливает алгоритм в ожидании события, записанного в иконе «Простой ввод». Кружок остается пустым

- Отсутствие зрительного шума: ДРАКОН-схемы обеспечивают простое и интуитивно понятное представление алгоритмов, что происходит благодаря отсутствию лишних графических элементов, отвлекающих внимание и не несущих смысла
- Силуэт: ДРАКОН-схема, разделенная на ветки (смысловые части) для удобства понимания алгоритма. Применяется в не примитивных ДРАКОН-схемах (10 и более икон)
- Чем правее — тем позже: ветка, нарисованная правее, работает позже всех веток, находящихся левее нее. Алгоритм, нарисованный согласно правилу «чем правее — тем позже», считается хорошим, эргономичным. Схемы, где это правило нарушается, объявляются плохими

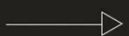
- Сферы применения: ДРАКОН-схемы могут быть использованы для различных видов алгоритмов и процессов, от простых до сложных. Часто ДРАКОН-схемы используются в космической отрасли, в бизнес-процессах, в медицине и в обучении

2. Диаграммы деятельности UML

Диаграмма деятельности — это один из типов диаграмм в языке UML (Unified Modeling Language), графическое представление процессов и задач, выполняемых в рамках определенного проекта или деятельности.

Ключевые моменты:

- В языке UML десятки разных элементов. Основные из элементов:

КЛАСС  Набор объектов со схожими свойствами	ИНТЕРФЕЙС  Набор доступных операций для объекта	ЮЗКЕЙС  Набор действий, которая система может выполнить или не выполнить	СВЯЗЬ  Показывает, что объекты связаны между собой
КОМПОНЕНТ  Большая часть системы, например, файл или библиотека	УЗЕЛ  Ещё более крупная часть системы. Как правило, содержит компоненты, например, сервер, на котором запущена программа	ВЗАИМОДЕЙСТВИЯ  Обмен информацией между компонентами системы	ЗАВИСИМОСТЬ  При изменении одного элемента изменяется и другой, соединенный с ним стрелкой такого типа
СОСТОЯНИЕ  Особая конфигурация системы, через которую она проходит. Например, фаза вычислений, во время которой программа не отвечает на другие запросы	ПАКЕТ  Объединение нескольких элементов диаграммы в группу по определенному критерию	ЗАМЕТКА  Небольшой комментарий к отдельным элементам диаграммы	АГРЕГАЦИЯ  Целевой элемент, часть исходного элемента
ОБОБЩЕНИЕ  Исходный элемент. Частная разновидность другого, более общего элемента, как объект и класс в программировании			

- Поток управления: Диаграммы действий UML сосредоточены на потоке управления от одного действия к другому, который является потоком, управляемым внутренней обработкой
- Ориентация: Диаграммы действий UML ориентированы на объекты, а не на процессы
- Состояния и переходы: Каждое состояние соответствует выполнению операции, а переход в следующее состояние происходит только после завершения этой операции

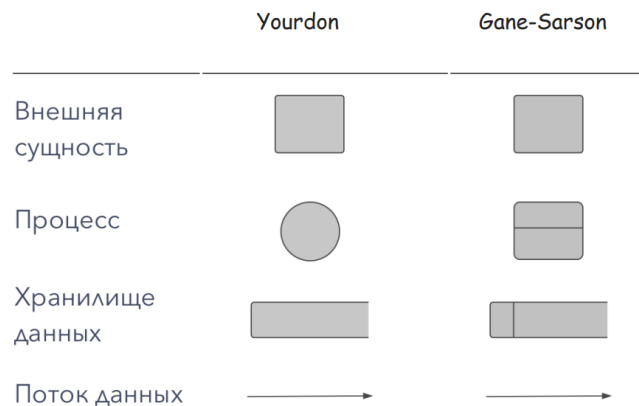
- Сферы применения: Используется для анализа и оптимизации бизнес-процессов, планирования работы, принятия решений и управления проектами
- Анализ процессов: Помогает выявить проблемы и определить области для улучшения процессов, что в свою очередь приводит к повышению эффективности и результативности деятельности

3. Диаграммы потоков данных (DFD)

Диаграмма потоков данных (data flow diagram, DFD) — методология графического структурного анализа, описывающая внешние по отношению к системе источники и адресаты данных, логические функции, потоки данных и хранилища данных, к которым осуществляется доступ.

Ключевые моменты:

- Символы: Исторически синтаксис этой нотации применяется в двух вариантах — Йордана (Yourdon) и Гейна-Карсона (Gane-Sarson).



- Диаграмма не должна разрастаться до безграничных размеров: Некоторые данные можно сгруппировать и дать пояснение уже в техническом задании. Это облегчит восприятие и сделает диаграмму более читаемой.
- Уровни детализации: DFD может быть представлен на нескольких уровнях — от общего (контекстная диаграмма) до детализированного (декомпозиция процессов), что позволяет постепенно углубляться в детали системы.
- Каждый процесс должен иметь хотя бы один вход и один выход: Смысл процессов здесь заключается в обработке именно данных, а потому процесс должен получить данные и отдать куда-то после обработки
- Должна поступать новая информация: Процесс обработки данных должен иметь внешнюю входящую стрелку (данные от внешней сущности). Для того, чтобы любой подобный процесс начал работать, мало использовать данные из хранилища, должна поступить новая информация для последующей обработки
- Все связи идут через процессы: Нет смысла просто перемещать данные из одного места в другое (связь двух хранилищ стрелкой). Данные поступают для того, чтобы производились какие-то действия. А это возможно только посредством обработки (процесса)

- Результат должен куда-то передаваться: Все процессы должны быть связаны либо с другими процессами, либо с другими хранилищами данных. Процессы не существуют сами по себе

4. Р-схемы

Р-схема — это метод визуального представления алгоритмов с использованием графических элементов, который позволяет формально записывать логику.

Ключевые моменты:

- Символы:

Наименование	Обозначение и его содержание	Функция
1. Вершина	 Окружность диаметром не менее 2 мм	Вершина Р-схемы
2. Вершина специальная	 Левая и правая круглые скобки на расстоянии не менее 1 мм	Выделение вершины Р-схемы
3. Дуга	а)  б)  Горизонтальная линия со стрелкой а) справа б) слева	Направленное соединение двух вершин Р-схемы
4. Дуга специальная	 Две горизонтальные линии, отстоящие друг от друга на расстоянии 0,8 – 3 мм	Специальное соединение двух вершин Р-схемы
5. Линия соединительная	 Вертикальная линия	Соединение по вертикали указанных выше элементов Р-схемы
6. Комментарий	 Пунктирная вертикальная (вверх или вниз) и горизонтальная (влево или вправо) линии, оканчивающиеся квадратной скобкой, за которой следует текст комментария.	Связь между элементом Р-схемы и текстом комментария.

- Иерархичность: Позволяет разбивать сложные алгоритмы на более простые подзадачи, что облегчает их понимание и реализацию
- Единая графическая форма: Не зависящая от языков программирования, единая графическая форма для записи и алгоритмов, и программ, и данных, и процессов на всех стадиях и этапах их разработки
- Граф, нагруженный по дугам: В Р-схемах логическое выражение записывается над дугой, а оператор присваивания и оператор процедуры — под дугой
- Горизонтальное расположение схем: Удобно при работе с широкоформатными мониторами.
- Применение: Широко используется в программировании, обучении, а также в бизнес-аналитике для описания процессов и принятия решений

5. Диаграммы Нэсси-Шнейдермана

Диаграмма Нэсси-Шнейдермана — это инструмент для визуализации структуры программных систем и их взаимосвязей.

Ключевые моменты:

- Символы:



- **Графическая структура:** Отображает компоненты системы и их взаимодействия в виде графа, что позволяет наглядно увидеть связи между элементами.
- **Возможность пошаговой детализации:** Изначально диаграмма представляет собой один прямоугольник (исходная задача), затем в нём рисуется структура управления, в которой имеется несколько прямоугольников (подзадач исходной задачи), и далее с каждым прямоугольником (подзадачей) может быть проделана та же операция.
- **Идентификация узких мест:** Помогает выявить узкие места и потенциальные проблемы в системе, что способствует оптимизации процессов и улучшению производительности.
- **Компактность:** Запись более компактная, чем блок-схемы, за счёт отсутствия стрелок между элементами.